

[Nome dell'Università]
[Nome del Dipartimento/Facoltà]
[Corso di Laurea Magistrale in ...]

Replica e Studio di Ablation di MetaSTGAT

per il Controllo Semaforico Adattivo

Relatore

[Nome e Cognome]

Candidato

[Nome e Cognome]

Anno Accademico [20XX/20XX]

Sommario

Il controllo semaforico su una rete di intersezioni è un problema di coordinamento distribuito: la fase scelta a un incrocio influenza, con pochi secondi di ritardo, il traffico che raggiunge gli incroci vicini. Le soluzioni classiche, a tempi fissi o reattive sulla sola pressione locale (MaxPressure), non tengono conto di questa dipendenza spaziale in modo esplicito. MetaSTGAT (Wang et al., 2022) affronta il problema con un agente di reinforcement learning per intersezione, coordinato da una *graph attention network* i cui pesi sono generati dinamicamente da un modulo di meta-learning condizionato su caratteristiche locali della rete stradale.

Questo lavoro replica da zero l’architettura e la procedura di training di MetaSTGAT su un ambiente costruito su CITYFLOW, con reti a griglia di tre dimensioni (4×4 , 5×5 , 6×6) e traffico generato parametricamente, incluso uno scenario con un’arteria a densità asimmetrica usato per misurare l’equità del controllo tra direzioni di traffico diverse. Ogni differenza introdotta rispetto alla procedura originale, dalla ricompensa alla struttura del replay buffer, è isolata da un sistema di preset di ablation e motivata singolarmente invece di essere presentata come un insieme indistinto di miglioramenti.

I risultati sperimentali raccolti finora confrontano le baseline classiche (Fixed-Time, MaxPressure) con diverse varianti del modello proposto. Un risultato non atteso emerge dallo sweep sul peso α della penalità anti-starvation della ricompensa: rimuoverla del tutto ($\alpha = 0$) non solo fa collassare l’equità direzionale verso i valori di MaxPressure, ma peggiora anche il travel time medio e massimo rispetto alle varianti con $\alpha > 0$, escludendo un semplice compromesso tra equità e throughput. Il confronto tra il modello completo e la variante senza BPTT e burn-in sulla componente ricorrente conferma inoltre il contributo di questo meccanismo, con un effetto proporzionalmente maggiore sulla generalizzazione a topologie mai viste che sulle configurazioni di training.

Al momento della stesura, lo studio di ablation copre due dei sei preset proposti e l’estensione dell’architettura a meccanismi spaziali alternativi alla *graph attention* (una convoluzione grafica e un meccanismo di propagazione a lungo raggio

ispirato a SONAR) è in fase di progettazione: questo sommario e le conclusioni saranno aggiornati man mano che quel lavoro sarà completato.

Indice

Sommario	i
Notazione e Acronimi	vii
1 Introduzione	1
1.1 Contributo di questo lavoro	2
1.2 Struttura della tesi	3
2 Background	4
2.1 Reinforcement Learning	4
2.2 Graph Neural Network	7
2.3 Traffic Signal Control come problema	8
3 TSC e Modelli	10
3.1 Formalizzazione del problema	10
3.2 Ambiente di simulazione	12
3.2.1 Struttura del roadnet e mappatura delle fasi	13
3.2.2 Gestione di verde, giallo e tutto-rosso	14
3.2.3 Campo visivo	15
3.2.4 Dataset e generazione parametrica	16
3.2.5 Semplificazioni dichiarate	19
3.3 Baseline classiche	19
3.4 Architettura MetaSTGAT	20
3.4.1 State Encoder duale	21
3.4.2 Meta-knowledge learner	21
3.4.3 Meta-GAT	21
3.4.4 Meta-LSTM	22
3.4.5 Testa Q-value	23
3.4.6 Dal paper al modello Pro: differenze motivate	23
3.5 Procedura di training e differenze dal paper originale	25

4	Confronto	27
4.1	Metriche di valutazione	27
4.2	Protocollo sperimentale	29
4.3	Risultati: baseline e peso anti-starvation	31
4.4	Equità direzionale: il ruolo di α	33
4.5	Studio di ablation: risultati preliminari	35

Elenco delle figure

- 4.1 Confronto su `config_4x4_100m_train1` (in-distribuzione): Fixed-Time, MaxPressure, MetaSTGAT Pro ($\alpha = 0.5/0.2$) e `ablation_temporal`. Manca solo $\alpha = 0.1$ (in training): figura da rigenerare includendolo, insieme ai preset di ablation restanti, a valle di questo capitolo. . . . 33
- 4.2 Confronto sulla configurazione di generalizzazione più difficile (6×6 , densità di picco), stessi sei controllori di Figura 4.1. 34

Elenco delle tabelle

3.1	Le 8 fasi semaforiche e i movimenti che ciascuna abilita.	14
3.2	Le cinque fasce del flusso di training "giornata lavorativa" (totale: 4.620-4.728 veicoli a seconda della variante seed, §3.2.4).	17
3.3	Ruolo di ciascun gruppo di configurazioni nella pipeline sperimentale.	19
4.1	Preset di ablation e sottosistema isolato da ciascuno.	31
4.2	Travel time medio, massimo e percentuale di completamento, mediati sulle 3 configurazioni di training (in-distribuzione).	32
4.3	Come Tabella 4.2, mediata sulle 7 configurazioni di test (generalizza- zione a topologia e densità mai viste in training).	32
4.4	Attesa massima direzionale (N/S e W/E), mediata sulle configurazio- ni di training e di test rispettivamente.	34
4.5	MetaSTGAT Pro ($\alpha = 0.5$) contro <code>ablation_temporal</code> (BPTT e burn-in disattivati), stesso α	35

Notazione e Acronimi

Acronimi

Acronimo	Significato
MDP	Markov Decision Process
RL	Reinforcement Learning
DQN	Deep Q-Network
PER	Prioritized Experience Replay
BPTT	Backpropagation Through Time
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
TSC	Traffic Signal Control
SMK	Spatial Meta-Knowledge (learner)
TMK	Temporal Meta-Knowledge (learner)
CS	Cooperation Spatial (modulo Meta-GAT)
CST	Cooperation Spatial-Temporal (modulo Meta-GAT)

Notazione principale

Simbolo	Significato
$\mathcal{S}, \mathcal{A}, P, R, \gamma$	Stati, azioni, transizione, ricompensa, sconto (MDP, §2.1)
π, V^π, Q^π	Politica, funzione valore-stato, funzione valore-azione
θ, θ^-	Pesi della rete online e della target network
τ	Coefficiente di soft update (Polyak averaging)
α (in RL)	Esponente di prioritizzazione del PER
α (in questo lavoro)	Peso della penalità anti-starvation nella ricompensa <code>custom</code> (§3.1)
ε	Probabilità di esplorazione (ε -greedy)
s_i^t, a_i^t, r_i^t	Stato, azione, ricompensa dell'intersezione i al tempo t
n_i^t, w_i^t, p_i^t	Veicoli per corsia, attesa massima per corsia, fase corrente (componenti di s_i^t)
d_0	Dimensione dello stato in ingresso (32 avanzato, 20 paper)
d_1	Dimensione nascosta del modello (default 64)
H, d_h	Numero di teste di attenzione e dimensione per testa ($d_h = d_1/H$)
e_i^t, e_j^t	Embedding temporale e spaziale prodotti dal Dual State Encoder
x_i^t, h_i^t, c_i^t	Output, stato nascosto e stato di cella del Meta-LSTM
$\text{SMK}(i), \text{TMK}(i)$	Meta-embedding spaziale e temporale dell'intersezione i
$z_{\text{CS}}(i), z_{\text{CST}}(i)$	Output dei moduli Meta-GAT spaziale e spazio-temporale
$\overline{tt}, tt_{\max}$	Travel time medio (ufficiale) e massimo
<code>wait_maxNS/EW</code>	Attesa massima direzionale (equità, §4.1)

Capitolo 1

Introduzione

Un incrocio semaforizzato decide, ogni pochi secondi, quale flusso di veicoli può muoversi e quale deve fermarsi. Su una singola intersezione isolata la scelta ottima è quasi banale: dare la precedenza a chi ha più traffico in attesa. Su una rete di decine di intersezioni la stessa scelta smette di essere locale: la fase scelta a un incrocio determina, con un ritardo di pochi secondi, il volume di veicoli che arriverà all'incrocio successivo. Il controllo semaforico su rete è quindi un problema di coordinamento distribuito prima ancora che un problema di ottimizzazione puntuale, e i sistemi storicamente più diffusi lo ignorano quasi del tutto.

Il controllo a tempi fissi (*fixed-time*) applica una sequenza di fasi di durata pre-stabilita, calcolata offline sulla base di stime di traffico medie, e non reagisce in alcun modo alle condizioni osservate in tempo reale: è la soluzione più semplice da implementare, ed è quella tipicamente in uso nella maggior parte degli impianti reali. Il controllo reattivo, di cui MAX-PRESSURE (Varaiya, 2013) è l'esempio più studiato in letteratura, introduce una regola locale che seleziona a ogni istante la fase con la pressione istantanea più alta (differenza tra veicoli in ingresso e in uscita sulle corsie servite da quella fase). MAX-PRESSURE garantisce proprietà di stabilità della rete nel suo insieme, ma la sua natura puramente greedy non offre alcuna garanzia di equità: un'intersezione posta su un'arteria molto trafficata può favorire sistematicamente la direzione dominante, lasciando la direzione minoritaria in attesa per periodi arbitrariamente lunghi: un comportamento che questo lavoro osserva empiricamente e discute nel Capitolo 3.

Il controllo appreso tramite *reinforcement learning* (RL) propone un'alternativa diversa: invece di una regola scritta a mano, una politica π viene addestrata a massimizzare una ricompensa che codifica l'obiettivo di rete (tipicamente throughput e tempo di viaggio) osservando ripetutamente le conseguenze delle proprie azioni in simulazione. Applicato a una singola intersezione il problema si riduce a

un MDP standard; applicato a una rete di intersezioni, il problema diventa naturalmente multi-agente, con osservabilità parziale locale (ogni intersezione osserva solo il proprio intorno) e la necessità di un meccanismo che permetta a un agente di tenere conto di ciò che accade agli agenti vicini. Le *graph neural network* (GNN) offrono esattamente questo meccanismo: rappresentando la rete stradale come un grafo, con le intersezioni come nodi e le strade come archi, uno strato di attenzione o convoluzione grafica propaga informazione tra nodi vicini a ogni passo, permettendo a un'intersezione di condizionare la propria decisione sullo stato dei suoi vicini immediati senza richiedere una politica centralizzata.

MetaSTGAT (Wang et al., 2022) combina questi due ingredienti con un terzo: invece di condividere gli stessi pesi tra tutte le intersezioni della rete, un modulo di *meta-learning* genera dinamicamente i pesi dello strato di attenzione grafica e dello strato ricorrente a partire da caratteristiche locali dell'intersezione (pressione sulle corsie, distanza dai vicini, storico della coda), così che nodi topologicamente diversi (un incrocio a T, un incrocio a quattro vie su un'arteria, un incrocio periferico con poco traffico) possano essere serviti da trasformazioni diverse pur restando un unico modello addestrato end-to-end. Gli autori originali riportano, su quattro dataset sintetici e due reali, una riduzione del travel time medio compresa tra l'8,24% e il 19,30% rispetto a CoLight, un metodo di riferimento a livello di grafo per lo stesso problema: un miglioramento sostanziale ma non uniforme tra scenari, un dettaglio che da solo suggerisce quanto il guadagno di un meccanismo come questo dipenda dalle caratteristiche specifiche della rete su cui viene misurato, non solo dall'architettura in sé.

1.1 Contributo di questo lavoro

Questa tesi replica da zero l'architettura e la procedura di training di MetaSTGAT, la sottopone a uno studio di ablation sistematico e la confronta con le baseline classiche del controllo semaforico. Nel dettaglio:

- un ambiente di simulazione multi-intersezione costruito su CITYFLOW (Zhang et al., 2019), con reti stradali a griglia di dimensione 4×4 , 5×5 e 6×6 , otto fasi semaforiche per intersezione e una formulazione MDP dettagliata nel Capitolo 3;
- una generazione parametrica dei dati di traffico, con tre livelli di densità (flusso di riferimento, *flat* e *peak*) e uno scenario dedicato di generalizzazione che intro-

duce un'arteria asimmetrica per verificare se un modello di controllo produce uno squilibrio direzionale sistematico quando il traffico non è simmetrico;

- un sistema di ablation a preset che isola singolarmente ogni differenza tra l'implementazione e la procedura descritta nel paper originale (dal meccanismo di replay all'architettura dello stato, dalla funzione di perdita alla ricompensa), attivabile tramite flag da riga di comando e organizzato in sei configurazioni che vanno dalla replica fedele del paper al modello più avanzato di questo lavoro;
- un protocollo di valutazione che riporta sempre il caso peggiore osservato, non solo la media: tempo di viaggio dei veicoli che non hanno completato il percorso entro la fine della simulazione, code del tempo di viaggio (percentili, massimo, deviazione standard) ed equità direzionale tra le corsie di una stessa intersezione, incluse le attese ancora in corso al termine dell'episodio.

Al momento della stesura di questo capitolo, l'estensione dell'architettura verso meccanismi spaziali alternativi alla *graph attention* (una convoluzione grafica standard, GCN, e un meccanismo di propagazione a lungo raggio ispirato a SONAR) è ancora in fase di progettazione e non è inclusa nei risultati qui presentati. Il Capitolo 3 ne introduce comunque la motivazione, e il confronto sperimentale a parità di resto dell'architettura è rimandato a un aggiornamento successivo di questo lavoro.

1.2 Struttura della tesi

Il Capitolo 2 introduce il background necessario a leggere i capitoli successivi senza assumere familiarità pregressa con nessuno dei tre campi coinvolti: reinforcement learning, graph neural network e controllo semaforico come problema. Il Capitolo 3 formalizza il problema affrontato come MDP, descrive l'ambiente di simulazione e le baseline classiche, e presenta l'architettura di MetaSTGAT insieme a ogni differenza introdotta rispetto al paper originale, motivata caso per caso. Il Capitolo 4 descrive il protocollo sperimentale e riporta i risultati ottenuti, incluso lo studio di ablation. Il Capitolo ?? discute i risultati alla luce della domanda di ricerca, dichiara i limiti dello studio e indica il lavoro rimasto per la seconda parte del progetto.

Capitolo 2

Background

2.1 Reinforcement Learning

Un problema di decisione sequenziale in ambiente stocastico si formalizza, nella trattazione ormai standard del campo (Sutton and Barto, 2018), come processo decisionale di Markov (MDP), una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ dove $\mathcal{S}$ è lo spazio degli stati, $\mathcal{A}$ lo spazio delle azioni, $P(s' | s, a)$ la probabilità di transizione (la proprietà di Markov richiede che dipenda solo dallo stato e dall'azione corrente, non dalla storia precedente), $R(s, a)$ la ricompensa attesa e $\gamma \in [0, 1)$ il fattore di sconto, che pesa meno le ricompense più lontane nel tempo e garantisce che il ritorno resti finito anche su un orizzonte infinito. Un agente segue una politica $\pi(a | s)$, una distribuzione di probabilità sulle azioni condizionata allo stato corrente; il suo obiettivo è massimizzare il ritorno atteso $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, la somma scontata delle ricompense future a partire dall'istante t . La funzione valore-stato $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ e la funzione valore-azione $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$ misurano rispettivamente quanto ritorno atteso si ottiene seguendo π da uno stato, o da una coppia stato-azione; soddisfano l'equazione di Bellman, che esprime il valore di uno stato in funzione del valore atteso degli stati successivi (un'equazione ricorsiva, non una formula chiusa):

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')] \quad (2.1)$$

Per la politica ottima π^* , quella che massimizza $V^\pi(s)$ per ogni stato simultaneamente, vale l'analoga equazione di Bellman di ottimalità:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

Il Q-learning tabellare approssima Q^* per iterazione a punto fisso dell'Equazione 2.2, ma richiede una tabella con una entry per ogni coppia (s, a) : impraticabile non appena $\mathcal{S}$ diventa continuo o anche solo troppo grande da enumerare, com'è il caso di un vettore di stato che descrive occupazione delle corsie e fase corrente di un'intersezione.

Deep Q-Network. Mnih et al. (2015) sostituiscono la tabella con una rete neurale $Q(s, a; \theta)$ e stabilizzano il training di questo approssimatore, altrimenti instabile per la correlazione tra transizioni successive e per il bersaglio mobile della propria stessa stima, con due accorgimenti: un *replay buffer* da cui campionare mini-batch di transizioni (s, a, r, s') in ordine indipendente dalla traiettoria in cui sono state raccolte (rompendo la correlazione temporale tra transizioni consecutive, che violerebbe altrimenti l'assunzione di campioni indipendenti su cui si basa la discesa del gradiente stocastico), e una *target network* $Q(\cdot; \theta^-)$ i cui pesi sono copiati periodicamente da quelli della rete online, usata per calcolare il bersaglio $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ invece che con la rete che si sta aggiornando nello stesso istante: senza questo accorgimento, il bersaglio si sposterebbe a ogni passo di ottimizzazione insieme ai pesi che lo generano, un problema di stabilità noto come *moving target*.

Esplorazione ε -greedy. Un agente che seguisse sempre e solo l'azione a valore Q stimato più alto non raccoglierebbe mai esperienza sulle conseguenze di azioni alternative, rischiando di convergere su una politica sub-ottima semplicemente perché non ha mai osservato di meglio. La strategia più comune per bilanciare sfruttamento (*exploitation*, scegliere l'azione ritenuta migliore) ed esplorazione (*exploration*, raccogliere informazione su azioni meno note) è ε -greedy: a ogni passo di decisione, con probabilità ε l'agente sceglie un'azione uniformemente a caso, e con probabilità $1 - \varepsilon$ sceglie $\arg\max_a Q(s, a; \theta)$. Il valore di ε tipicamente decresce nel corso del training, da un valore iniziale vicino a 1 (esplorazione quasi totale, quando la stima di Q non è ancora affidabile) verso un valore finale piccolo ma non nullo (la politica appresa guida quasi sempre la scelta, ma non smette mai del tutto di raccogliere esperienza su alternative).

Estensioni usate in questo lavoro. Il DQN di base ammette diverse estensioni indipendenti, ciascuna delle quali corregge una specifica fonte di instabilità o inefficienza; la procedura di training descritta nel Capitolo 3 le adotta tutte, motivandole caso per caso rispetto al paper di riferimento:

- Il **Double DQN** (van Hasselt et al., 2016) disaccoppia la selezione dell'azione dalla sua valutazione, usando la rete online per scegliere l'azione migliore e la target network solo per valutarla: $y = r + \gamma Q(s', \operatorname{argmax}_{a'} Q(s', a'; \theta); \theta^-)$. Questo riduce il bias sistematico di sovrastima del DQN standard, che usa lo stesso massimo sia per scegliere sia per valutare l'azione.
- Il **Prioritized Experience Replay** (PER, Schaul et al., 2016) sostituisce il campionamento uniforme dal replay buffer con un campionamento proporzionale all'errore di differenza temporale δ_i della transizione: priorità $p_i = |\delta_i| + \varepsilon$, probabilità di campionamento $P(i) = p_i^\alpha / \sum_k p_k^\alpha$. Per correggere il bias introdotto da un campionamento non uniforme, ogni transizione è pesata nella loss da un peso di importance sampling $w_i = (N \cdot P(i))^{-\beta} / \max_j w_j$.
- La **Huber loss** (o *smooth L1*) sostituisce l'errore quadratico medio con una funzione quadratica vicino allo zero e lineare lontano da esso, meno sensibile agli outlier prodotti da transizioni con errore di stima molto grande:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \delta \\ \delta \left(|y - \hat{y}| - \frac{1}{2}\delta \right) & \text{altrimenti} \end{cases} \quad (2.3)$$

- Il **soft update** (*Polyak averaging*) aggiorna la target network con una media esponenziale invece di una copia periodica secca: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$, con τ tipicamente piccolo (nell'ordine di 10^{-2}), rendendo il bersaglio più stabile passo per passo.
- Quando la rete Q include un componente ricorrente (una LSTM, come nell'architettura del Capitolo 3), campionare transizioni singole dal replay buffer pone un problema ulteriore: lo stato ricorrente (h, c) usato durante il training viene inizializzato a zero, mentre durante la raccolta dati proveniva dalla storia reale dell'episodio: un disallineamento noto come *hidden state staleness*. Kapturowski et al. (2019) propongono di campionare intere sequenze di lunghezza L invece di transizioni singole e di dedicare i primi passi della sequenza (*burn-in*) esclusivamente a ricostruire uno stato ricorrente plausibile, senza calcolare gradiente, prima di eseguire la *backpropagation through time* (BPTT) sui passi restanti.

2.2 Graph Neural Network

Molti problemi non hanno una struttura a griglia regolare (come un'immagine) o sequenziale (come una frase), ma una struttura a grafo: un insieme di entità (nodi) connesse da relazioni (archi) la cui topologia è essa stessa informazione rilevante. Una rete stradale è un esempio naturale: le intersezioni sono nodi, le strade che le collegano sono archi, e lo stato di un'intersezione (quanti veicoli attende, quale fase è attiva) è influenzato da ciò che accade alle intersezioni immediatamente a monte e a valle. Le GNN generalizzano le operazioni di convoluzione e attenzione a questo tipo di dato, propagando informazione tra nodi collegati da un arco.

Graph Convolutional Network. Kipf and Welling (2017) propagano l'informazione aggregando, per ogni nodo, una media pesata delle rappresentazioni dei propri vicini (normalizzata sul grado dei nodi coinvolti), seguita da una trasformazione lineare condivisa e da una non linearità: è il meccanismo più semplice, con lo stesso peso per ogni arco del vicinato di un nodo.

Graph Attention Network. Veličković et al. (2018) sostituiscono il peso fisso della GCN con un coefficiente di attenzione appreso, diverso per ogni coppia di nodi collegati. Per un nodo i con vicinato $\mathcal{N}(i)$, embedding di input h_i , matrice di proiezione condivisa W e vettore di attenzione $\vec{a}$:

$$e_{ij} = \text{LeakyReLU}(\vec{a}^\top [Wh_i \parallel Wh_j]), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (2.4)$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j \right) \quad (2.5)$$

dove $\parallel$ indica concatenazione. Con K teste di attenzione indipendenti, gli output si concatenano nei layer intermedi e si mediano nel layer finale, seguendo lo schema multi-head introdotto per l'attenzione da Vaswani et al. (2017). A differenza della GCN, il coefficiente α_{ij} permette a un nodo di pesare diversamente i propri vicini in base al loro contenuto, non solo alla loro posizione nel grafo: rilevante per un'intersezione con vicini di importanza molto diversa (un'arteria principale contro una strada secondaria).

Pesi generati da meta-learning. Sia la GCN sia la GAT, nella loro formulazione originale, condividono gli stessi pesi (W , $\vec{a}$) su tutti i nodi del grafo: un'assunzione ragionevole quando i nodi sono sufficientemente omogenei, meno quando, come in

una rete stradale reale, la topologia locale di ogni nodo (numero di strade entranti, presenza di un'arteria, posizione periferica o centrale) può differire da nodo a nodo. Un *hypernetwork* affronta questo problema generando i pesi di un layer non come parametri fissi appresi direttamente, ma come output di una seconda rete condizionata su caratteristiche locali del nodo: il Capitolo 3 descrive nel dettaglio come MetaSTGAT applichi questo principio sia allo strato di attenzione grafica sia allo strato ricorrente.

Propagazione a lungo raggio. Sia la GCN sia la GAT propagano informazione a un solo passo di distanza (un salto nel grafo) per layer: raggiungere un vicino a distanza k richiede impilare k layer, con un costo di parametri crescente e un rischio noto di *over-smoothing* (le rappresentazioni dei nodi diventano indistinguibili quando la profondità cresce troppo). Meccanismi più recenti, tra cui SONAR (paper allegato a questo progetto, [Trenta et al., 2025](#)), affrontano il problema della propagazione a lungo raggio con un principio diverso, ispirato a sistemi dinamici oscillatori piuttosto che all'impilamento di layer a pesi indipendenti: il dettaglio di questo meccanismo e la sua applicazione al controllo semaforico sono rimandati a un lavoro successivo (si veda la nota nel Capitolo 1).

2.3 Traffic Signal Control come problema

Il controllo semaforico (*Traffic Signal Control*, TSC) si può classificare in tre famiglie in ordine crescente di complessità.

Controllo a tempi fissi. Una sequenza di fasi di durata predeterminata, calcolata offline su stime di traffico medie, si ripete ciclicamente senza mai osservare lo stato reale della rete. È robusto (non può fallire per un sensore guasto) ma per costruzione non reattivo: la stessa sequenza viene eseguita indipendentemente da un incidente, un evento, o una semplice fluttuazione oraria del traffico non prevista in fase di progettazione.

Controllo reattivo. Una regola locale osserva lo stato corrente dell'intersezione (tipicamente il numero di veicoli in coda per corsia) e sceglie l'azione che ottimizza un criterio istantaneo. MAX-PRESSURE ([Varaiya, 2013](#)) ne è l'esponente più studiato: modellando il movimento dei veicoli come una rete di code "store-and-forward" (i veicoli si accumulano su una corsia finché non vengono serviti, poi si spostano sulla coda a valle), definisce la pressione di una fase ϕ come la differenza tra veicoli sulle corsie che quella fase servirebbe in ingresso e in uscita, e sceglie sempre l'azione a pressione massima. Il risultato teorico centrale dell'articolo originale (il suo Teore-

ma 2) è che Max-Pressure è *massimamente stabilizzante*: se esiste una qualunque politica di controllo capace di mantenere stabile (code non divergenti nel tempo) un dato pattern di domanda sulla rete, allora anche Max-Pressure la mantiene stabile, e di conseguenza massimizza il throughput della rete nel lungo periodo. Ottiene questa garanzia usando solo informazione locale (la lunghezza delle code sulle corsie adiacenti all'intersezione) e senza conoscere a priori il volume di domanda in ingresso all'intero sistema. È però per costruzione miope: non pianifica alcun compromesso su un orizzonte più lungo di un passo, e la garanzia di stabilità/throughput non dice nulla sulla distribuzione delle attese tra le diverse direzioni servite da un'intersezione, un aspetto di equità che questo lavoro esamina esplicitamente (Capitolo 4).

Controllo appreso. Il reinforcement learning permette di ottimizzare direttamente un obiettivo di rete (tempo di viaggio, throughput) invece di una regola surrogata come la pressione istantanea, al costo di richiedere una fase di addestramento in simulazione. Applicato a una rete di più intersezioni, il problema è naturalmente un MDP decentralizzato e parzialmente osservabile: ogni intersezione osserva solo il proprio stato locale (le proprie corsie, la propria fase), ma le sue azioni influenzano, con un ritardo di pochi secondi, lo stato delle intersezioni immediatamente vicine attraverso il flusso di veicoli che le attraversa. Un agente per intersezione che non condivide alcuna informazione con i propri vicini ignora questa dipendenza; una GNN, propagando informazione lungo gli archi del grafo stradale a ogni passo di decisione, la rende esplicita nella rappresentazione di stato di ogni agente.

Simulazione. Valutare una politica di controllo semaforico su una rete reale non è praticabile durante lo sviluppo: richiede un simulatore a eventi discreti che riproduca la dinamica del traffico (posizione dei veicoli, codifica delle corsie, transizioni di fase) con un dettaglio sufficiente perché i risultati siano informativi ma con un costo computazionale che permetta migliaia di episodi di addestramento in tempi ragionevoli. CITYFLOW (Zhang et al., 2019) è il simulatore usato in questo lavoro: rispetto ad alternative più diffuse come SUMO, privilegia la velocità di simulazione rispetto al realismo microscopico del singolo veicolo, un compromesso adatto quando l'obiettivo primario è addestrare un agente su un numero elevato di episodi piuttosto che riprodurre con precisione millimetrica la dinamica di un incrocio specifico. Il Capitolo 3 descrive nel dettaglio come questo lavoro configura l'ambiente CityFlow e formalizza il problema come MDP.

Capitolo 3

TSC e Modelli

3.1 Formalizzazione del problema

Il controllo semaforico su una rete di M intersezioni si formalizza come un MDP multi-agente con osservabilità locale: ogni intersezione i osserva uno stato s_i^t , sceglie un'azione a_i^t e riceve una ricompensa r_i^t , mentre la topologia della rete (quali intersezioni sono collegate da una strada) definisce un grafo su cui l'informazione viene propagata a livello di modello, non di ambiente.

Stato. Ogni intersezione ha $N_LANES = 12$ corsie in ingresso (tre per ciascuna delle quattro direzioni cardinali) e $N_PHASES = 8$ fasi semaforiche selezionabili. Lo stato è la concatenazione di tre componenti:

$$s_i^t = [n_i^t \parallel w_i^t \parallel p_i^t] \in \mathbb{R}^{d_0} \quad (3.1)$$

dove $n_i^t \in [0, 1]^{12}$ è il numero di veicoli per corsia (normalizzato, saturato a 30 veicoli), $w_i^t \in [0, 1]^{12}$ è il tempo di attesa massimo per corsia (normalizzato, saturato a 100s) e $p_i^t \in \{0, 1\}^8$ è la fase corrente in codifica one-hot. La componente w_i^t non è presente nella formulazione originale del paper, che usa $d_0 = 20$; la variante di questo lavoro con w_i^t inclusa porta $d_0 = 32$ (§3.4.6, punto A1). Nella modalità avanzata, n_i^t conta solo i veicoli entro $VISION_CUTOFF_M$ dal semaforo (§3.2); nella modalità fedele al paper la visibilità è sull'intera corsia.

Azione. $a_i^t \in \{0, \dots, 7\}$ seleziona una delle otto fasi; la fase di tutto-rosso interna alla transizione (§3.2) non è mai un'azione disponibile all'agente. Un vincolo anti-starvation invalida la fase corrente come azione se è già stata selezionata due volte

consecutive, forzando la rotazione:

$$\mathcal{A}_i^t = \{0, \dots, 7\} \setminus \{a_i^{t-1}\} \quad \text{se } a_i^{t-1} = a_i^{t-2}, \quad \mathcal{A}_i^t = \{0, \dots, 7\} \text{ altrimenti} \quad (3.2)$$

Questo vincolo si applica solo all'agente RL: MaxPressure (§3.3) resta una pura argmax senza vincoli, fedele all'implementazione di riferimento. Il numero due non è arbitrario: nelle primissime fasi di training, quando la politica è ancora vicina al caso e non ha alcuna preferenza appresa, un agente senza questo vincolo tende a bloccarsi sulla prima fase che produce un ritorno anche solo leggermente positivo, ripetendola indefinitamente (un "verde fisso" degenerato); vietare la ripetizione oltre la seconda volta consecutiva rompe questo ciclo senza impedire a una fase di restare attiva per un tempo ragionevole quando è davvero la scelta migliore per due passi di fila.

Questo vincolo va tenuto distinto, concettualmente, dalla metrica di equità direzionale introdotta nel Capitolo 4 (§4.1): sono due meccanismi indipendenti, che agiscono in punti diversi della pipeline. La maschera sulle azioni agisce *a monte*, sulla scelta della singola azione, e si limita a vietare tre ripetizioni consecutive della stessa fase; l'equità direzionale è invece una misura osservazionale *a valle*, calcolata sull'intero episodio, che non impone alcun vincolo ma si limita a registrare quanto a lungo attende la direzione sfavorita. Un modello può rispettare rigorosamente il vincolo delle due ripetizioni (non sceglie mai la stessa fase tre volte di fila) e mostrare comunque una forte disparità tra le direzioni N/S e W/E, se torna sistematicamente sulla fase che serve la direzione dominante non appena il vincolo delle due ripetizioni glielo permette di nuovo: evitare il verde fisso non è la stessa cosa che garantire un'alternanza equa.

Ricompensa. Due modalità, selezionabili con il flag `reward_mode`. La modalità `custom` (di default in questo lavoro):

$$r_i^t = \text{clip}\left(\frac{\text{Passed}_i^t - \text{Incoming}_i^t - \alpha \cdot \text{MaxRedWait}_i^t - \text{Penalty}_i^t}{100}, -20, 5\right) \quad (3.3)$$

dove Passed_i^t è il numero di veicoli che hanno lasciato la finestra visiva dell'intersezione tra t e $t+1$, Incoming_i^t il numero di veicoli entrati nella finestra visiva al tempo t , $\text{Penalty}_i^t = 50$ se $\text{Passed}_i^t = 0 \wedge \text{Incoming}_i^t > 0$ (un verde che non fa passare nessuno) e 0 altrimenti, e α l'iperparametro anti-starvation (sweep condotto in questo lavoro su $\alpha \in \{0.0, 0.2, 0.5\}$). La modalità `paper` riproduce fedelmente Wang et al. (2022,

Eq. 1):

$$r_i^t = -\frac{P_i^t}{100}, \quad P_i^t = \text{Incoming}_i^t - \text{Outgoing}_i^t \quad (3.4)$$

La divisione per 100 in entrambe le modalità è una riscalatura numerica aggiunta per la stabilità del training, non presente nella formula originale del paper ma priva di effetto sulla politica ottima indotta (è un fattore di scala costante sulla ricompensa).

Nota terminologica: il termine P_i^t dell'Equazione 3.4 è un conteggio aggregato per intersezione (ingresso meno uscita nella finestra visiva) e non va confuso con la pressione classica di Varaiya usata da MaxPressure (§3.3, Equazione 3.6), che è definita per movimento (coppia corsia di ingresso/corsia di uscita) e sull'intera corsia, senza alcun cutoff di visibilità: sono due formule distinte che condividono solo il nome informale di "pressione".

Grafo. Le M intersezioni sono nodi di un grafo $G = (V, E)$; un arco $(i, j) \in E$ esiste se una strada collega direttamente i e j nel roadnet. Quando un nodo ha più di $\text{num_neighbors} = 4$ vicini diretti, l'insieme è troncato ai 4 più vicini per distanza euclidea; sulle reti a griglia usate in questo lavoro (§3.2) i nodi interni hanno già esattamente 4 vicini diretti, per cui il troncamento incide solo sui nodi con connettività superiore alla griglia regolare, se presenti.

Un dettaglio rilevante per l'interpretazione dei risultati di generalizzazione (§4): né lo stato s_i^t né le feature spaziali e temporali che alimentano SMK/TMK (§3.4) contengono l'identità o la posizione assoluta del nodo i all'interno della griglia, solo conteggi di veicoli sulle proprie corsie e uno storico recente; il Meta-GAT, inoltre, condivide la stessa rete generatrice di pesi ($W_h^{(i)}$, §3.4.3) su tutti i nodi, condizionandola sul meta-embedding locale ma non su un identificatore del nodo. Una politica appresa non ha quindi, in linea di principio, alcun modo diretto di "memorizzare" che un'arteria si trova sempre sulla stessa riga della griglia: può solo imparare a reagire a un pattern osservato di pressione persistente, ovunque questo si presenti. Se questa aspettativa regga effettivamente alla prova sperimentale, e non solo in linea di principio, è una domanda aperta che l'uso di tre righe diverse per l'arteria nelle varianti di training (§3.2) è pensato per non lasciare priva di un tentativo di verifica.

3.2 Ambiente di simulazione

L'ambiente è costruito su CITYFLOW (Zhang et al., 2019), un simulatore di traffico microscopico open source con motore in C++ e binding Python, pensato esplicitamente per il controllo semaforico multi-intersezione via reinforcement learning su

reti di dimensione grande. I suoi stessi autori lo motivano con un limite pratico del simulatore open source più diffuso in letteratura, SUMO, non sufficientemente scalabile a reti di grandi dimensioni e a volumi di traffico elevati da permettere un addestramento RL su larga scala in tempi ragionevoli: CITYFLOW riporta, sulle proprie reti sintetiche di riferimento (30×30 intersezioni, decine di migliaia di veicoli, 8 thread), una velocità di simulazione circa 20-25 volte superiore a SUMO. Il compromesso alla base di questo guadagno è un livello di dettaglio fisico del singolo veicolo inferiore a quello di un simulatore come SUMO: adatto quando l'obiettivo primario è addestrare un agente su migliaia di episodi piuttosto che riprodurre con precisione millimetrica la dinamica di un incrocio specifico. La simulazione avanza a passi discreti di un secondo (`engine.next_step()`): non esiste un modo nativo di avanzare di più secondi in una sola chiamata, per cui ogni passo di decisione dell'agente (§3.1) richiede più chiamate consecutive al motore, gestite internamente dal wrapper.

3.2.1 Struttura del roadnet e mappatura delle fasi

La topologia di ogni rete è definita in un file di roadnet: una lista di intersezioni (posizione, e un'indicazione se l'intersezione è reale e semaforizzata oppure "virtuale", cioè un punto ai bordi della griglia dove le strade escono dalla mappa senza alcun semaforo) e una lista di strade, segmenti diretti tra due intersezioni con un numero di corsie ciascuno. Per ogni intersezione reale, il roadnet elenca inoltre le fasi semaforiche disponibili, ciascuna definita come l'insieme dei movimenti (coppie corsia-in/corsia-out) abilitati in quella fase: comandare il semaforo significa semplicemente indicare al motore l'indice della fase da attivare.

Ogni intersezione di questo lavoro ha $N_LANES = 12$ corsie in ingresso modellate (quattro direzioni cardinali, tre corsie ciascuna: sinistra, dritto, destra) e $N_PHASES = 8$ fasi verdi selezionabili, elencate in Tabella 3.1 con la nomenclatura N/S/E/O (Nord/Sud/Est/Ovest) più T/L (*Through*, dritto, e *Left*, sinistra) usata anche nella Figura 1 del paper originale.

Una scelta di realismo dichiarata esplicitamente nel codice: la svolta a destra non è mai automaticamente abilitata (il cosiddetto *free-right*, una semplificazione comune in altri lavori sul controllo semaforico), ma attende la fase che la autorizza esplicitamente, come qualunque altro movimento. Quando un'intersezione ai margini della griglia ha fisicamente meno di quattro strade collegate, le corsie mancanti sono modellate come corsie fittizie sempre a zero veicoli, in modo che lo stato s_i^t (§3.1) abbia sempre la stessa dimensione indipendentemente dalla posizione dell'intersezione nella griglia.

Tabella 3.1: Le 8 fasi semaforiche e i movimenti che ciascuna abilita.

Fase	Nome	Movimenti abilitati
0	NTST	Nord diritto+destra, Sud diritto+destra
1	NLSL	Nord sinistra, Sud sinistra
2	NTNL	Nord sinistra+diritto+destra (solo Nord)
3	STSL	Sud sinistra+diritto+destra (solo Sud)
4	WTET	Ovest diritto+destra, Est diritto+destra
5	WLEL	Ovest sinistra, Est sinistra
6	ETEL	Est sinistra+diritto+destra (solo Est)
7	WTWL	Ovest sinistra+diritto+destra (solo Ovest)

3.2.2 Gestione di verde, giallo e tutto-rosso

CITYFLOW, nella sua forma nativa, non ha alcun concetto di "giallo" o di fisica del semaforo ambra: una fase è solo un insieme binario di movimenti abilitati o disabilitati, senza stati intermedi. Qualunque comportamento di tipo giallo/tutto-rosso deve quindi essere costruito esplicitamente sopra questa primitiva, definendo fasi aggiuntive nel roadnet e pilotandole da codice passo per passo. Questo lavoro lo fa nel modo seguente. Ogni passo di decisione dura $STEP_TIME = GREEN_TIME + YELLOW_TIME + RED_TIME = 10 + 3 + 2 = 15$ secondi simulati: la fase scelta dall'agente resta attiva per i primi $GREEN_TIME + YELLOW_TIME = 13$ secondi (il giallo è trattato come un prolungamento del verde: il movimento resta abilitato per tutti e 13 i secondi, senza un vincolo di sicurezza aggiuntivo diverso dal tempo trascorso), seguiti da un'ulteriore fase, aggiunta appositamente al roadnet e mai selezionabile dall'agente (indice 8, fuori dallo spazio delle azioni $\{0, \dots, 7\}$), che disabilita esplicitamente ogni movimento su ogni intersezione per $RED_TIME = 2$ secondi.

Questo meccanismo non era presente nella prima versione di questo lavoro: inizialmente i 5 secondi finali di ogni passo venivano eseguiti sotto la stessa identica fase verde dei primi 10, senza una seconda chiamata di comando al semaforo, il che equivaleva funzionalmente a 15 secondi di verde continuo senza alcun istante di tutto-rosso. La finestra di rischio reale era stretta (un'intersezione larga circa 20 metri si attraversa in pochi secondi, per cui il problema riguardava solo i veicoli entrati negli ultimissimi istanti di verde) ma genuina: un veicolo ancora dentro l'incrocio al cambio di fase poteva in linea di principio sovrapporsi a un veicolo di un movimento conflittuale appena abilitato dalla fase successiva. La correzione, applicata in modo meccanico e uniforme a tutte le intersezioni semaforizzate di tutti i roadnet del progetto, è stata verificata non solo leggendo il codice ma ispezionando riga per riga il replay grezzo prodotto dal motore per una singola intersezione campione: lo stato

delle corsie, per un ciclo completo, passa da $['r', 'g', 'g']$ (verde più giallo, fase invariata) a $['r', 'r', 'r']$ (tutte le corsie rosse simultaneamente, il tutto-rosso) per tornare a $['g', 'r', 'r']$ (la fase successiva, verde su una corsia diversa): conferma diretta che il tutto-rosso è genuinamente simulato, non solo contabilizzato a parte, e che le fasi verdi si alternano davvero invece di ripetere sempre lo stesso schema. Poiché questo comportamento è identico per ogni modello confrontato in questo lavoro (Pro, ogni preset di ablation, Fixed-Time, MaxPressure passano tutti dallo stesso ciclo di step), il confronto relativo tra modelli resta valido; cambiano leggermente, in modo uniforme, i valori assoluti: due dei quindici secondi per ciclo (circa il 13%) sono ora tempo morto reale invece che verde esteso.

3.2.3 Campo visivo

Nella modalità avanzata dello stato (§3.1), i veicoli sono contati solo entro VISION_CUTOFF_M dal semaforo:

$$\begin{aligned}\text{VISION_CUTOFF_M} &= (\text{GREEN_TIME} + \text{YELLOW_TIME}) \cdot v_{\text{urb}} \\ &= 13 \cdot 11.1 \approx 144.3 \text{ m}\end{aligned}\tag{3.5}$$

con $v_{\text{urb}} = 11.1 \text{ m/s}$ ($\approx 40 \text{ km/h}$) velocità urbana di riferimento: rappresenta la distanza che un veicolo può effettivamente percorrere nella finestra temporale utile di un'azione (i 13 secondi di verde più giallo, non i 15 secondi dell'intero passo, dato che nei 2 secondi di tutto-rosso nessun veicolo avanza), non un valore arbitrario scelto per convenienza. Se le costanti GREEN_TIME o YELLOW_TIME cambiassero, il cutoff si aggiornerebbe automaticamente di conseguenza. Il cutoff è specifico allo stato dell'agente RL: la pressione classica usata da MaxPressure (§3.3) non ne è soggetta, per fedeltà alla sua definizione originale, che non prevede alcun raggio di visibilità limitato.

Un dettaglio implementativo corretto nel corso di questo lavoro, incluso qui come esempio del tipo di verifica applicata a ogni componente dell'ambiente: la distanza usata per calcolare il cutoff su ciascuna corsia va sottratta alla lunghezza reale della corsia stessa, che inizialmente era approssimata con la distanza punto-a-punto tra i centri delle due intersezioni collegate. CITYFLOW tronca però la distanza percorsa riportata per un veicolo alla larghezza fisica dell'intersezione a *entrambe* le estremità della corsia: un veicolo lascia la corsia già alcuni metri prima del punto centrale dichiarato nel roadnet. Usare la distanza punto-a-punto sovrastimava quindi la lunghezza reale della corsia e rendeva, di conseguenza, il cutoff di visibilità più restrittivo di quanto derivato in teoria; la correzione sottrae esplicitamente la

larghezza di entrambe le intersezioni coinvolte prima di applicare il cutoff.

3.2.4 Dataset e generazione parametrica

Ogni roadnet e ogni flusso di traffico usato in questo lavoro è generato parametricamente e in modo interamente deterministico da uno script dedicato, non raccolto da fonti esterne: questo permette di controllare esattamente quali variabili cambiano tra una configurazione e l'altra (topologia, densità, presenza di un'arteria) e di rigenerare l'intero dataset da zero se un parametro va rivisto, cosa che è avvenuta più volte nel corso del progetto (si veda più sotto la calibrazione della densità su 5×5 e 6×6).

Topologie e calibrazione dell'onda verde. Tre griglie regolari, 4×4 (16 intersezioni reali), 5×5 (25) e 6×6 (36), usate per misurare la generalizzazione a topologie di dimensione diversa da quella di training. Sulla griglia 4×4 esistono due varianti di spaziatura tra gli incroci, usate per isolare l'effetto di un fenomeno specifico: un'onda verde. Poiché tutte le intersezioni di questo lavoro cambiano fase in modo sincrono (non esiste una sfasatura temporale tra un incrocio e il successivo), l'unico modo per ottenere un'onda verde, cioè far sì che un veicolo che parte all'inizio di un verde arrivi al semaforo successivo esattamente quando questo torna verde, è calibrare la distanza fisica tra gli incroci sulla velocità di marcia e sulla durata del ciclo: alla velocità di riferimento $v_{\text{urb}} = 11.1$ m/s, un ciclo completo di 15 secondi corrisponde a $11.1 \times 15 \approx 166.5$ metri, la distanza reale a cui sono calibrate le reti etichettate "100m" in questo lavoro (l'etichetta indica un parametro di generazione interno, non la distanza reale, che tiene conto anche di un offset geometrico fisso). Una seconda variante della griglia 4×4 , etichettata "200m" (260 metri reali), mantiene una spaziatura non calibrata, usata come configurazione di test aggiuntiva per misurare quanto un modello allenato sulla rete con onda verde regge quando la spaziatura cambia; non viene mai usata per il training. Le griglie 5×5 e 6×6 esistono solo nella variante calibrata, dato che il loro scopo è testare la generalizzazione alla dimensione della rete, non l'effetto dell'onda verde (già isolato sulla griglia 4×4).

Densità di traffico. Per ciascuna topologia sono generati flussi a tre livelli di densità: un flusso costante per l'intera durata dell'episodio (*flat*), un flusso a tre fasi (*peak*, un terzo dell'episodio a intensità bassa, un terzo a intensità raddoppiata, un terzo di nuovo bassa) e, solo per il training, un flusso "giornata lavorativa" descritto più sotto. Ogni episodio dura $\text{maxStep} = 1800$ secondi simulati, corrispondenti a $1800/\text{STEP_TIME} = 120$ passi di decisione per intersezione. La densità di traffico

per ciascuna configurazione non è stata fissata arbitrariamente: una prima versione, calibrata assumendo che una griglia più grande assorbisse proporzionalmente meglio il traffico grazie a un minor numero di intersezioni di bordo, si è rivelata sbagliata quando verificata sperimentalmente con MaxPressure (un controllore reale, seppur semplice, il cui throughput riflette quindi quanto la rete regge il traffico assegnato, non solo quanto è inefficiente il controllore): a parità di veicoli per intersezione, le griglie più grandi assorbivano il traffico *peggio*, non meglio, verosimilmente perché la congestione si accumula lungo percorsi che attraversano più intersezioni in sequenza. La densità di ciascuna configurazione di questo lavoro è stata quindi ricalibrata empiricamente fino a raggiungere un throughput confrontabile (90-94%) con MaxPressure su tutte e tre le topologie, non assunta teoricamente corretta a tavolino.

Il flusso di training "giornata lavorativa". A differenza delle configurazioni di validazione e test, che usano una densità costante o a picco singolo, il flusso usato per il training mappa una giornata lavorativa di 24 ore su un episodio di 1800 secondi simulati, in cinque fasce di durata e intensità diverse (Tabella 3.2): due fasce a intensità bassa (mattina presto e notte), due fasce di punta (pendolari mattutini e fine giornata lavorativa) e una fascia centrale a intensità sostenuta per la maggior parte dell'episodio. Questa non-stazionarietà interna a un singolo episodio è intenzionale: impedisce al modello di limitarsi a imparare una singola densità di traffico fissa, cosa che una configurazione di addestramento a densità costante permetterebbe.

Tabella 3.2: Le cinque fasce del flusso di training "giornata lavorativa" (totale: 4.620-4.728 veicoli a seconda della variante seed, §3.2.4).

Fascia	Intervallo simulato	Passi di decisione	Veicoli/secondo
Flat basso (mattina presto)	0-525s	0-35	1.2
Peak basso (pendolari mattutini)	525-675s	35-45	2.5
Flat alto (ore diurne)	675-1200s	45-80	3.5
Peak alto (fine giornata lavorativa)	1200-1350s	80-90	4.2
Flat basso (notte)	1350-1800s	90-120	1.2

Tre varianti seed e arteria Est-Ovest. Due accorgimenti, introdotti per evitare un rischio concreto di overfitting metodologico. Primo: poiché CITYFLOW è deterministico a parità di flusso e seed, usare sempre lo stesso file di flusso per il

training esporrebbe il modello, episodio dopo episodio, alla sequenza *esatta* di veicoli (stessi istanti di generazione, stessi percorsi), con il rischio che il modello impari quella sequenza specifica invece di una politica di controllo genuinamente generalizzabile. Il training di questo lavoro cicla quindi tra tre varianti dello stesso profilo di densità (stessa Tabella 3.2, un episodio di training appartiene sempre a una sola variante, alternata deterministicamente per numero di episodio), generate con seed diversi che cambiano il rumore applicato al peso di ciascun percorso e agli istanti esatti di generazione dei veicoli, lasciando invariato il profilo aggregato (quanti veicoli, quando, con quale forma). Le configurazioni di validazione e di test restano invece a seed singolo, dato che il loro scopo è offrire un metro di paragone stabile, non introdurre ulteriore varianza.

Secondo: un controllo preliminare sul flusso di training originale ha mostrato che il carico di traffico tra le diverse righe della griglia era già naturalmente quasi uniforme (entro il $\pm 10\%$), un'asimmetria trascurabile che non avrebbe permesso di verificare una domanda specifica, cioè se il termine anti-starvation della ricompensa *custom* (Equazione 3.3) fornisca ai modelli RL un vantaggio di equità che MaxPressure, per costruzione, non ha. È stata quindi introdotta deliberatamente un'arteria Est-Ovest a densità di traffico maggiore delle strade trasversali: ogni percorso che attraversa un segmento orizzontale sulla riga designata vede il proprio peso, prima della normalizzazione, moltiplicato per un fattore 3, una *ridistribuzione* dello stesso budget di veicoli (le altre righe ricevono proporzionalmente meno), non un'aggiunta di traffico complessivo. Per non lasciare non verificata l'ipotesi, discussa in §3.1, che una politica appresa possa generalizzare a un'arteria posta su una riga qualsiasi invece di specializzarsi sulla posizione osservata durante il training, le tre varianti seed hanno l'arteria su tre righe diverse della griglia; le configurazioni di validazione e di tutte le 7 configurazioni di test la mantengono invece fissa sulla stessa riga, per restare un benchmark stabile tra un modello e l'altro.

Split training/validazione/test. Tabella 3.3 riassume l'uso di ciascuna configurazione. Le tre varianti di training non sono mai usate per la selezione del modello finale o per il confronto tra modelli: quel ruolo spetta a una configurazione di validazione indipendente (§3.5), mentre le rimanenti 7 configurazioni (griglia 4×4 a 200m, 5×5 e 6×6 , ciascuna a densità *flat* e *peak*) non sono mai viste né in training né in validazione, riservate esclusivamente alla misura di generalizzazione riportata nel Capitolo 4.

Tabella 3.3: Ruolo di ciascun gruppo di configurazioni nella pipeline sperimentale.

Ruolo	Configurazioni
Training	3 varianti seed della griglia 4×4 a 100m con flusso "giornata lavorativa", cicliche un episodio alla volta
Validazione	Griglia 4×4 a 100m, densità <i>flat</i> , seed singolo: usata solo per scegliere tra i due checkpoint candidati a fine training (§3.5)
Test	Le 3 configurazioni di training stesse (prestazione in-distribuzione) più le 7 configurazioni rimanenti mai viste (generalizzazione a topologia, spaziatura e densità)

3.2.5 Semplificazioni dichiarate

Per completezza, alcune semplificazioni dell'ambiente vanno dichiarate esplicitamente invece di lasciarle implicite nel codice:

- Il giallo è trattato come un prolungamento del verde, non come uno stato fisico distinto: una scelta dichiarata, non equivoca dopo l'introduzione del tutto-rosso reale discussa sopra.
- CITYFLOW non simula la fisica di collisione punto per punto tra veicoli di movimenti diversi all'interno del poligono dell'intersezione; la sicurezza della transizione di fase è garantita a livello di simulazione dei movimenti abilitati e disabilitati (nessun veicolo nuovo entra su un movimento rosso), non da un motore fisico di collisione.
- Il campo visivo a cutoff di distanza e il proxy di congestione basato sul conteggio di veicoli per corsia (non sulla velocità media, non disponibile per singola corsia in CITYFLOW) sono approssimazioni dichiarate, non un tentativo di modellare la fisica del traffico in ogni dettaglio.

3.3 Baseline classiche

Fixed-Time. Cicla le 8 fasi in ordine fisso $(0, 1, \dots, 7, 0, \dots)$, cambiando fase a ogni passo di decisione (15s): non osserva mai lo stato della rete, non ha alcun parametro da configurare oltre all'ordine del ciclo, e non richiede training. La durata di ogni fase nel ciclo è deliberatamente identica a quella di ogni altro controllore di questo lavoro, RL o reattivo: un ciclo più lungo o più corto introdurrebbe un fattore di scala arbitrario nel confronto, rendendo impossibile distinguere un vantaggio dovuto alla strategia di controllo da uno dovuto solo a una cadenza diversa. Fixed-

Time rappresenta, in questo lavoro, il limite superiore di travel time che qualunque strategia informata, reattiva o appresa, dovrebbe poter migliorare (§4).

Max-Pressure. Implementazione fedele al riferimento LibSignal (Varaiya, 2013), un controllore reattivo, senza parametri appresi e senza necessità di training: a ogni passo di decisione, per ogni intersezione, calcola la pressione di ciascuna delle 8 fasi e seleziona quella a pressione massima. La pressione di una fase ϕ è la somma, su tutti i movimenti (coppia corsia di ingresso/corsia di uscita) abilitati da ϕ , della differenza tra veicoli sulla corsia di ingresso e veicoli sulla corsia di uscita, contati sull'intera corsia (nessun cutoff di visibilità):

$$P(i, \phi) = \sum_{(\ell_{\text{in}}, \ell_{\text{out}}) \in \text{movimenti}(\phi)} \left(n(\ell_{\text{in}}) - n(\ell_{\text{out}}) \right) \quad (3.6)$$

L'azione scelta è $\phi^*(i) = \operatorname{argmax}_{\phi} P(i, \phi)$ (a parità di pressione, l'indice di fase più basso, per stabilità); nessun vincolo di equità o anti-starvation è applicato, a differenza dell'agente RL (§3.1), e nessuna maschera sulle azioni consecutive: MaxPressure può, in linea di principio, scegliere la stessa fase per un numero arbitrario di passi consecutivi, se quella resta la fase a pressione più alta. Alcune implementazioni di riferimento (incluso LibSignal) prevedono un tempo minimo di permanenza su una fase prima di poterla cambiare, necessario quando l'agente può essere interrogato a una cadenza più fine di quella minima di sicurezza; nell'ambiente di questo lavoro ogni decisione dura già $\text{STEP_TIME} = 15$ secondi per costruzione (§3.2), quindi non esiste una cadenza più fine a cui l'agente venga interrogato, e questo vincolo aggiuntivo risulta già implicitamente soddisfatto senza bisogno di reimplementarlo esplicitamente.

3.4 Architettura MetaSTGAT

MetaSTGAT (Wang et al., 2022) elabora lo stato s_i^t di ogni intersezione attraverso cinque componenti in sequenza, prima di produrre i valori Q per le 8 azioni disponibili. La descrizione che segue è stata verificata riga per riga sul codice sorgente del modello (`src/models/`), non trascritta dalla sola notazione del paper: dove il paper è ambiguo (in particolare per Meta-GAT e Meta-LSTM, §3.4.3-3.4.4), è indicata esplicitamente quale delle interpretazioni possibili il codice implementa.

3.4.1 State Encoder duale

Lo stato $s_i^t \in \mathbb{R}^{d_0}$ ($d_0 = 32$ o 20 , §3.1) attraversa due MLP a due strati con la stessa architettura ma pesi e inizializzazioni indipendenti:

$$e_i^t = \text{ReLU}(W_{21} \text{ReLU}(W_{11}s_i^t + b_{11}) + b_{21}) \quad (3.7)$$

$$e_j^t = \text{ReLU}(W_{22} \text{ReLU}(W_{12}s_i^t + b_{12}) + b_{22}) \quad (3.8)$$

entrambi in $\mathbb{R}^{d_1}$ ($d_1 = 64$ di default). e_i^t (branch temporale) alimenta il Meta-LSTM (§3.4.4); e_j^t (branch spaziale) è la query di entrambi i moduli Meta-GAT (§3.4.3).

3.4.2 Meta-knowledge learner

Due MLP a due strati (stessa architettura dello state encoder, con un'attivazione tanh finale opzionale che normalizza l'embedding in $[-1, 1]$, disattivabile con il flag `-no-tanh-meta`) apprendono una rappresentazione compatta di caratteristiche locali dell'intersezione, usata a valle solo per generare i pesi di Meta-GAT e Meta-LSTM, mai come input diretto del Q-value:

$$\text{SMK}(i) = f_{\text{smk}}(x_i^{\text{smk}}) \in \mathbb{R}^{28 \rightarrow 64} \quad (3.9)$$

$$\text{TMK}(i) = f_{\text{tmk}}(x_i^{\text{tmk}}) \in \mathbb{R}^{72 \rightarrow 64} \quad (3.10)$$

dove il vettore di ingresso spaziale $x_i^{\text{smk}} \in \mathbb{R}^{28}$ concatena pressione per corsia [12], numero di veicoli per corsia [12] e distanza dai vicini [4]; il vettore temporale $x_i^{\text{tmk}} \in \mathbb{R}^{72}$ concatena la lunghezza di coda per corsia [12] e lo storico di n_i^t (§3.1) sugli ultimi 5 step [60]. La feature "location" menzionata nel paper originale per il TMK non è implementata in questo lavoro: un'omissione dichiarata, non una scelta motivata a posteriori (§3.4.6, punto C3).

3.4.3 Meta-GAT

Il modello usa due istanze dello stesso layer Meta-GAT, con input e meta-embedding diversi:

- **Modulo CS** (spaziale puro): $Q = K = V = e_j^t$, meta-embedding = $\text{SMK}(i)$, produce z_{CS} .
- **Modulo CST** (spazio-temporale): $Q = e_j^t$, $K = V = x_i^t$ (l'output del Meta-LSTM, §3.4.4), meta-embedding = $\text{TMK}(i)$, produce z_{CST} .

Per ciascun modulo, con $H = 4$ teste di dimensione $d_h = d_1/H = 16$: il meta-embedding genera, per ogni nodo i e ogni testa h , una matrice $W_h^{(i)} \in \mathbb{R}^{d_h \times d_h}$ e uno scalare $b_h^{(i)}$ (stessa rete a due strati usata per SMK/TMK: un layer condiviso con ReLU seguito da due proiezioni lineari indipendenti per W e b). Per ogni arco $(u \rightarrow i)$ del grafo (§3.1):

$$\varphi_h(i, u) = \frac{(W_h^{(i)} Q_h(i)) \cdot K_h(u) + b_h^{(i)}}{\sqrt{d_h}}, \quad \alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))} \quad (3.11)$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u), \quad z(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1} \quad (3.12)$$

Due punti dell’Equazione 3.11 risolvono un’ambiguità del paper (Eq. 14-17), che non specifica la dimensione di W né distingue $\sqrt{d_h}$ da $\sqrt{d_1}$: il codice applica $W_h^{(i)}$ alla query del nodo *ricevente* i (non del vicino u) prima del prodotto scalare con la key, e usa $\sqrt{d_h}$ (dimensione della singola testa) come fattore di scala, coerente con la formulazione standard dell’attenzione multi-head (Vaswani et al., 2017).

3.4.4 Meta-LSTM

Una rete a due strati genera, dal solo $\text{TMK}(i)$, i pesi $W_g^{(i)} \in \mathbb{R}^{2d_1 \times d_1}$ e i bias $b_g^{(i)} \in \mathbb{R}^{d_1}$ per ciascuno dei quattro gate $g \in \{\text{forget, input, output, cell}\}$ di una cella LSTM altrimenti standard:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(\left(W_{\{f, \iota, o, g\}}^{(i)} \right)^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f, \iota, o, g\}}^{(i)} \right) \quad (3.13)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t \quad (3.14)$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t) \quad (3.15)$$

con $[e_i^t \parallel h_i^{t-1}] \in \mathbb{R}^{2d_1}$ concatenazione dell’encoding corrente e dello stato nascosto precedente (ordine input-poi-hidden, coerente con la notazione del paper $\text{LSTM}(e_i^t, h_{t-1})$). L’output x_i^t (rinominato h_i^t per uniformità con la notazione LSTM standard) alimenta il modulo CST di Meta-GAT come K e V (§3.4.3). Il paper (Eq. 20) scrive $W_\Phi \in \mathbb{R}^{2D_h \times D_h}$ e $b_\Phi \in \mathbb{R}$ (un solo gate, bias scalare): il codice genera pesi e bias completi per tutti e quattro i gate, l’unica scelta compatibile con una LSTM funzionante (§3.4.6, categoria B).

3.4.5 Testa Q-value

I due embedding spaziali si concatenano e attraversano un singolo layer lineare, fedele all’Eq. 12 del paper:

$$q(s_i^t, \cdot) = W_q [z_{\text{CST}}(i) \parallel z_{\text{CS}}(i)] + b_q \in \mathbb{R}^8 \quad (3.16)$$

3.4.6 Dal paper al modello Pro: differenze motivate

Il modello Pro di questo lavoro non è un’architettura diversa dal paper, ma la stessa architettura con una serie di differenze puntuali rispetto alla Section 4.3 e all’Algorithm 1 originali, ciascuna motivata singolarmente e disattivabile indipendentemente tramite il sistema di ablation a preset (§3.5). Le differenze si dividono in tre categorie, a seconda di quanto la loro superiorità sia già stabilita o resti da verificare sperimentalmente.

A. Miglioramenti con motivazione tecnica solida.

- A1. *w_i^t nello stato* (§3.1): un segnale più informativo del solo conteggio veicoli, a parità di coda un’attesa di 90s è più urgente di una di 5s.
- A2. *Maschera azioni invalide*: previene il collasso su una singola fase (verde fisso), osservato come comportamento degenerativo nelle fasi iniziali di training senza vincolo.
- A3. *Double DQN*: riduce il bias di sovrastima del valore *Q* rispetto al DQN standard (van Hasselt et al., 2016).
- A4. *BPTT + burn-in* (sequenze $L = 4$, burn-in 2, R2D2-style Kapturowski et al., 2019): corregge l’hidden state staleness dello stato LSTM tra raccolta dati e training.
- A5. *Huber loss + pesi IS da PER*: più robusta agli outlier della MSE; il campionamento prioritario concentra il training sulle transizioni a errore TD più alto.
- A6. *Soft update* (Polyak, $\tau = 0.01$): target network più stabile di un hard-copy periodico.
- A7. *Gradient clipping* ($\|\nabla\| \leq 1.0$): stabilità numerica necessaria con un componente ricorrente nel grafo di computazione.

A8. *tanh finale nei meta-learner*: normalizza l’embedding generato in $[-1, 1]$ invece di lasciarlo libero.

A9. *Cutoff di visibilità* ($\approx 144\text{m}$): realismo fisico, non un numero arbitrario (§3.2).

A10. *Buffer PER a 4.800 sequenze* (vs 10.000 transizioni piatte del paper): calibrato deliberatamente sulla durata dell’episodio (≈ 40 episodi di storia con episodi da 1800s), non un ridimensionamento subito: la metrica di confronto corretta con il paper è la copertura in episodi, non il numero grezzo di transizioni.

B. Disambiguazioni di un paper sotto-specificato. La formula di attenzione di Meta-GAT (Eq. 14-17 del paper, §3.4.3) e le dimensioni dei pesi di Meta-LSTM (Eq. 20, §3.4.4) non sono deviazioni arbitrarie: il paper stesso lascia aperta più di un’interpretazione, e le scelte implementative descritte sopra sono quelle compatibili con un’architettura funzionante, non una tra più alternative egualmente valide.

C. Scelte da presentare con cautela. Un’ultima categoria di differenze non è ancora, a questo stadio del lavoro, un miglioramento dimostrato: nessun preset di ablation diverso da **pro** e **temporal** (§3.5) è stato allenato e testato al momento della stesura di questo capitolo.

C1. *Ricompensa riscritta* (Eq. 3.3 vs Eq. 3.4): la motivazione teorica è che una ricompensa di pura pressione può essere instabile, e che promuovere esplicitamente il throughput e penalizzare le attese prolungate produce un comportamento più bilanciato; il suo effetto netto su travel time e throughput, a parità di resto dell’architettura, è discusso nel Capitolo 4 alla luce dei risultati dell’ablation su **reward_mode**, non dato per acquisito qui.

C2. *Numero di episodi ed epsilon decay*: il vincolo di risorse (un solo processo, mai i 3 processi paralleli del paper) va dichiarato onestamente, ma non è di per sé un limite subito: il meccanismo di selezione del modello finale (§3.5) mostra che il training raggiunge il suo miglior risultato ben prima di esaurire il budget di episodi assegnato.

C3. *Feature "location" assente nel TMK*: omissione dichiarata, senza una motivazione cercata a posteriori.

3.5 Procedura di training e differenze dal paper originale

Il modello è addestrato con Double DQN (A3) su sequenze campionate da un replay buffer prioritizzato episodico (A10), con burn-in e BPTT (A4), Huber loss pesata dagli importance-sampling weight di PER (A5), soft update della target network (A6) e gradient clipping (A7); l'ottimizzatore è RMSprop, non Adam, per fedeltà alla Section 5.1 del paper originale: l'unica scelta architetturale/di training che questo lavoro non ha mai messo in discussione né reso configurabile via ablation, a differenza di ogni altra voce di questa sezione.

Iperparametri usati per il modello Pro (identici su tutte le varianti dello sweep su α): *learning rate* 10^{-3} , $\gamma = 0.85$, $d_1 = 64$, $H = 4$ teste, `num_neighbors` = 4, ε da 0.9 a 0.01 con decadimento esponenziale, batch size 20, buffer PER a 4.800 sequenze di lunghezza $L = 4$ con burn-in 2 (A4, A10). Ogni run di questo lavoro usa 100 episodi su un solo processo, contro i 200 episodi su 3 processi paralleli del paper (C2): il divario nel numero di episodi totali di raccolta dati è reale e dichiarato, ma, per la ragione discussa in C2, non equivale a un training interrotto prematuramente.

Ciclo di training. Prima dell'inizio del training vero e proprio, 10 episodi di *warm-up* (disattivabili) raccolgono transizioni con $\varepsilon = 1$ (azioni interamente casuali, non loggate come training vero) con il solo scopo di riempire il replay buffer fino a una soglia minima di 1.000 transizioni, sotto la quale il campionamento per l'aggiornamento dei pesi non è ancora avviato; con più configurazioni di training in ciclo, anche il warm-up alterna tra le varianti, così il buffer iniziale non proviene da una sola di esse. Nel training vero e proprio, ogni episodio appartiene a una sola variante di configurazione (alternata deterministicamente per numero di episodio, §3.2.4), al termine del quale l'agente esegue 100 aggiornamenti del gradiente: un numero fisso, indipendente dal numero di passi ambientali dell'episodio (120 con episodi da 1800 secondi), ereditato dal paper originale come iperparametro a sé stante, non ricalibrato quando la durata degli episodi è cambiata nel corso di questo lavoro. Il valore di ε decade una volta per episodio verso il suo minimo, raggiunto al 60% del numero totale di episodi pianificati per quel run (il tasso di decadimento non è quindi un numero fisso in assoluto, ma viene ricalcolato in funzione del numero effettivo di episodi richiesti): usare ε -greedy come strategia di esplorazione (§2.1) significa che, a ogni passo di decisione, l'agente sceglie un'azione casuale con probabilità ε e altrimenti l'azione a valore Q massimo secondo la stima corrente.

Oltre a questo decadimento continuo, un episodio ogni 10 viene eseguito intera-

mente a $\varepsilon = 1$ anche dopo che il valore minimo di ε è stato raggiunto (*esplorazione ciclica*, disattivabile), senza contribuire all’aggiornamento dei pesi in quello specifico episodio: l’obiettivo è continuare a inserire nel buffer esperienza che la politica ormai quasi deterministica non visiterebbe più spontaneamente, invece di lasciare che il replay buffer converga verso le sole traiettorie che la politica appresa preferisce. Questo lavoro ha esplicitamente valutato, e poi scartato, l’ipotesi che disattivare questo meccanismo una volta raggiunto il minimo di ε migliorasse la stabilità del training: un episodio interamente casuale a politica ormai quasi deterministica inserisce nel buffer transizioni potenzialmente fuori distribuzione con priorità massima di default, e un picco ripetuto nella loss subito dopo ogni occorrenza tardiva di questo meccanismo sembrava inizialmente confermare un effetto negativo; controllando però l’effetto per la variante di training attiva in quell’episodio (che cambia comunque a ogni episodio, indipendentemente dall’esplorazione ciclica), l’impatto sull’episodio successivo è risultato inconcludente, non sistematicamente negativo. Non abbastanza per modificare un meccanismo esistente e deliberatamente motivato: il comportamento originale è stato mantenuto.

Il sistema di ablation a preset (`pro/paper/ environment/temporal/rl_core/ replay_stability`) rende ciascuna delle differenze A1-A10, B e C1-C3 sopra individualmente disattivabile da riga di comando: il preset `paper` le disattiva tutte simultaneamente, riproducendo la procedura originale con lo stesso codice usato per il modello Pro. Il Capitolo 4 riporta i risultati sperimentali ottenuti, incluso lo studio di ablation costruito su questo stesso sistema di preset.

Capitolo 4

Confronto

4.1 Metriche di valutazione

Il reward di training (Equazioni 3.3-3.4) non è comparabile tra modalità **custom** e **paper**, per cui ogni confronto in questo capitolo usa esclusivamente metriche oggettive, calcolate allo stesso modo indipendentemente da come un modello è stato addestrato.

Travel time. Per ogni veicolo v , il tempo di viaggio è

$$tt_v = \begin{cases} t_{\text{arrivo}}(v) - t_{\text{spawn}}(v) & \text{se } v \text{ è arrivato entro la fine dell'episodio} \\ t_{\text{now}} - t_{\text{spawn}}(v) & \text{altrimenti (un lower bound del suo vero valore)} \end{cases} \quad (4.1)$$

La metrica ufficiale di questo lavoro, $\overline{tt}$, è la media di tt_v calcolata sull'intera popolazione di veicoli generati nell'episodio, non solo su quelli arrivati: include cioè deliberatamente i veicoli che non hanno completato il percorso entro la fine della simulazione. La ragione è evitare un bias di sopravvivenza concreto: un modello che congestiona la rete e lascia arrivare solo una minoranza di veicoli su percorsi rimasti liberi otterrebbe, calcolando la media solo sui veicoli arrivati, un risultato buono calcolato su un campione piccolo e non rappresentativo del comportamento reale della rete; nel caso limite in cui nessun veicolo arrivasse affatto, quella stessa metrica restituirebbe un travel time di 0 secondi, il punteggio migliore possibile per il comportamento peggiore immaginabile. Il travel time dei soli veicoli arrivati ($\overline{tt}_{\text{arr}}$) è riportato in ogni tabella di questo capitolo come dato informativo aggiuntivo, mai come criterio con cui scegliere tra modelli. Una terza variante, il travel time nativo calcolato direttamente dal motore CITYFLOW (che adotta anch'esso la convenzione "solo veicoli arrivati"), non è riportata in questo capitolo: serve solo per un confronto

diretto con un numero pubblicato in letteratura (il paper originale riporta un travel time di 430-470 secondi su una rete sintetica 4×4 , non direttamente comparabile con i valori di questo lavoro data la ricompensa e la procedura di training diverse, §3.4.6), non per confrontare i modelli di questo lavoro tra loro.

Coda del travel time. Oltre alla media, si riportano il massimo ($tt_{\max}$), la deviazione standard e i percentili $p_{90}/p_{95}/p_{99}$ del travel time dei veicoli arrivati: una media bassa può coesistere con una coda lunga di veicoli fortemente penalizzati, e nessuna delle due statistiche da sola distinguerebbe questo caso da una distribuzione uniformemente buona.

Throughput. Veicoli che hanno completato il percorso entro la fine dell'episodio, riportato sia in valore assoluto sia come percentuale sul totale dei veicoli generati (indicata $\%_{\text{compl}}$ nelle tabelle di questo capitolo). A differenza del travel time, il throughput non richiede alcuna convenzione sui veicoli non arrivati: un veicolo o ha completato il percorso o non lo ha fatto, senza casi intermedi da trattare.

Percentuali di scelta delle fasi. Per ciascuna delle 8 fasi (Tabella 3.1), la frazione di passi di decisione, su tutte le intersezioni e l'intero episodio, in cui quella fase è stata scelta. Non è di per sé una metrica di prestazione, nel senso che nessun valore è "migliore" in assoluto: è piuttosto un diagnostico della strategia appresa o applicata da un controllore. Un controllore che collassa sistematicamente su un piccolo sottoinsieme di fasi, ad esempio, sta probabilmente reagendo a uno squilibrio strutturale del traffico (una direzione più trafficata delle altre) piuttosto che alternare in modo bilanciato; MaxPressure, per la sua logica greedy sulla pressione istantanea, tende in pratica a preferire marcatamente le fasi che servono i movimenti "diritto" rispetto a quelle che servono le sole svolte a sinistra, dato che i primi accumulano pressione più rapidamente su una rete con volumi tipici.

Equità direzionale. Per ogni intersezione, la massima attesa storica *consecutiva* osservata su una corsia del gruppo N/S o del gruppo W/E (§3.1): consecutiva nel senso che il contatore si azzerava non appena il veicolo interessato si muove, non è quindi un'attesa cumulativa sull'intero viaggio del veicolo, ma la durata del più lungo singolo stallo mai osservato su una corsia di quel gruppo, in qualunque punto della rete. Il gruppo N/S raccoglie le corsie percorse da chi attraversa l'incrocio in direzione verticale (arrivando da Nord o da Sud), il gruppo W/E quelle percorse da chi lo attraversa in direzione orizzontale: è una classificazione per corsia fisica di approccio all'incrocio in un dato istante, non un'origine o una destinazione del

viaggio del veicolo. Il valore ufficiale, `wait_maxNS` e `wait_maxEW`, è il massimo di questa quantità su tutte le intersezioni della rete, e include gli stalli ancora in corso al termine dell'episodio (censura a destra), per lo stesso principio di non-esclusione del caso peggiore applicato al travel time: un veicolo fermo nello stesso punto dal momento in cui si è bloccato fino all'ultimo istante simulato non fa mai scattare, per definizione, un nuovo record *dopo* la fine dell'episodio, e una metrica che considerasse solo gli stalli conclusi lo farebbe sparire, premiando indirettamente chi lascia un veicolo bloccato per sempre invece di penalizzarlo come merita. Il caso non è ipotetico: durante lo sviluppo di questa metrica, un valore di `wait_maxNS = 1125` secondi riportato da MaxPressure è stato verificato ricostruendo la traiettoria del veicolo responsabile direttamente dal replay grezzo della simulazione (posizione per veicolo, secondo per secondo, un canale indipendente da qualunque contabilità interna del codice): il veicolo risultava fermo nello stesso identico punto dal secondo 679 fino al secondo 1799, l'ultimo istante simulato dell'intero episodio, uno stallone la cui vera durata resta sconosciuta perché la simulazione si è fermata, non perché il veicolo sia stato servito. Questa metrica non compare nella ricompensa di alcun modello: è puramente osservazionale, pensata per verificare se il termine anti-starvation della ricompensa `custom` (Equazione 3.3) produce un vantaggio reale rispetto a MaxPressure, che non ha alcuna nozione di equità, né di un vincolo simile alla maschera sulle azioni consecutive che regola invece l'agente RL (§3.1): i due meccanismi restano concettualmente indipendenti, e il rispetto dell'uno non implica il rispetto dell'altro. Una variante di questa metrica, che considera solo gli stalli conclusi entro la fine dell'episodio (esclude cioè la censura a destra discussa sopra), è calcolata e riportata come colonna secondaria in ogni tabella di riepilogo, ma non è mai usata come criterio ufficiale di confronto in questo capitolo, per lo stesso principio di non-esclusione del caso peggiore.

4.2 Protocollo sperimentale

Training, validazione, test. Ogni modello RL è addestrato in ciclo su tre varianti a seed diverso della stessa configurazione base (`config_4x4_100m_train`, `_s2`, `_s3`), con l'arteria Est-Ovest su una riga diversa in ciascuna (righe 1, 2, 0 rispettivamente, §3.2): un episodio di training appartiene sempre a una sola variante, alternata deterministicamente per numero di episodio. A fine training, due checkpoint candidati, l'ultimo episodio (`final_model.pth`) e l'episodio con il miglior travel time mai osservato durante il training (`best_model.pt`, non necessariamente l'ultimo), sono valutati per un episodio ciascuno su una configurazione di validazione

indipendente, `config_4x4_100m_6k_flat`, mai usata né in training né nel confronto finale tra modelli; il vincitore diventa `selected_model.pth`, il checkpoint cercato con priorità massima da ogni valutazione successiva (in sua assenza, la ricerca ripiega su `final_model.pth`, poi su `best_model.pt`, poi sul checkpoint periodico più recente disponibile). Ogni valutazione, sia in questa fase di selezione sia nella batteria di test descritta sotto, avviene a $\varepsilon = 0$: nessuna esplorazione residua, solo la politica appresa allo stato puro, dato che introdurre azioni casuali in fase di valutazione misurerebbe la qualità di una politica che non è quella che si intende effettivamente confrontare.

Batteria di test. Ogni modello, RL o baseline, è valutato sulle stesse 10 configurazioni: le 3 varianti di training stesse (per misurare la prestazione in-distribuzione) più 7 configurazioni mai viste in training (4x4 a 200m invece di 100m, 5×5 e 6×6 a densità sia *flat* sia *peak*), usate per misurare la generalizzazione a topologie e densità di traffico diverse. Testare ogni modello su entrambi i gruppi, non solo sulle configurazioni di generalizzazione, è la prassi adottata in questo lavoro dopo averla resa parte dell’orchestrazione standard (`main.py`).

Determinismo della simulazione. Con flusso di traffico e seed fissati nel file di configurazione e `laneChange=False`, CITYFLOW è deterministico bit-per-bit: a parità di modello e configurazione, ogni valutazione produce esattamente la stessa traiettoria. La valutazione standard usa quindi un solo episodio per configurazione (`n_eval=1`): ripeterlo non ridurrebbe alcuna varianza reale. Questo ha una conseguenza sull’interpretazione dei risultati di questo capitolo che va dichiarata esplicitamente: ogni numero riportato è l’esito esatto di un’unica realizzazione deterministica per coppia (modello, configurazione), non una media campionaria con un proprio errore standard. Le differenze tra modelli sullo stesso file di configurazione sono quindi confronti esatti, non stime soggette a rumore di campionamento; non è invece possibile, con i dati di questo lavoro, quantificare quanto un singolo risultato varierebbe se la stessa configurazione fosse ricampionata con un seed diverso. Le tre varianti di training (seed distinti sulla stessa distribuzione di traffico) sono lo strumento con cui questo lavoro approssima quella domanda, non la risolvono in senso statistico stretto.

Sistema di ablation a preset. Ogni preset (`-ablation`) imposta un gruppo coerente di flag, isolando un sottosistema alla volta rispetto al modello Pro:

Tabella 4.1: Preset di ablation e sottosistema isolato da ciascuno.

Preset	Reward	Cosa isola rispetto a Pro
pro	custom	(nessuno: modello di riferimento)
paper	paper	Replica fedele di Wang et al. (2022)
environment	paper	Ingegnerizzazione dell'MDP (visibilità, w_i^t , maschera)
temporal	custom	BPTT + burn-in sulla Meta-LSTM
rl_core	custom	Double DQN
replay_stability	custom	PER, Huber, soft update, grad clipping, warm-up, esplorazione ciclica

Per la riproducibilità: ogni modello di questo capitolo è ottenuto con lo stesso comando, a parità di configurazioni di training e di seed, cambiando solo il preset (e, per lo sweep descritto in §4.3, il peso α della ricompensa):

Listing 4.1: Comando di training, modello Pro con $\alpha = 0.2$.

```
python scripts/train.py \
  --config configs/config_4x4_100m_train1.json \
    configs/config_4x4_100m_train2.json \
    configs/config_4x4_100m_train3.json \
  --model MetaSTGAT --episodes 100 --seed 42 \
  --ablation pro --alpha 0.2 \
  --output results/metastgat_pro_0.2
```

4.3 Risultati: baseline classiche e peso anti-starvation

Questa sezione confronta sei controllori: le due baseline classiche (§3.3), tre varianti del modello Pro che differiscono solo per il peso α della penalità anti-starvation nella ricompensa `custom` (Equazione 3.3), e `ablation_temporal` (discusso separatamente in §4.5, dove il confronto rilevante è con Pro a parità di α). $\alpha = 0.1$ e i quattro preset di ablation restanti (`paper`, `environment`, `rl_core`, `replay_stability`) sono in fase di addestramento al momento della stesura di questo capitolo: i risultati che seguono si limitano ai modelli già valutati sulla batteria di test completa. Per Fixed-Time non sono disponibili $tt_{\max}$ e le statistiche di equità direzionale (calcolate solo a partire dall'11/9/2026, dopo che questo modello era già stato valutato): le colonne corrispondenti sono segnate "n.d.".

Fixed-Time è, come atteso, il controllore peggiore su ogni colonna in entrambi i regimi: non osservando mai lo stato della rete, stabilisce il limite superiore

Tabella 4.2: Travel time medio, massimo e percentuale di completamento, mediati sulle 3 configurazioni di training (in-distribuzione).

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	% _{compl}
Fixed-Time	339.3	n.d.	84.9
MaxPressure	161.9	1070.0	96.6
MetaSTGAT Pro ($\alpha = 0.5$)	205.5	800.0	93.9
MetaSTGAT Pro ($\alpha = 0.2$)	194.0	745.0	94.9
MetaSTGAT Pro ($\alpha = 0.0$)	211.2	1350.0	91.5

Tabella 4.3: Come Tabella 4.2, mediata sulle 7 configurazioni di test (generalizzazione a topologia e densità mai viste in training).

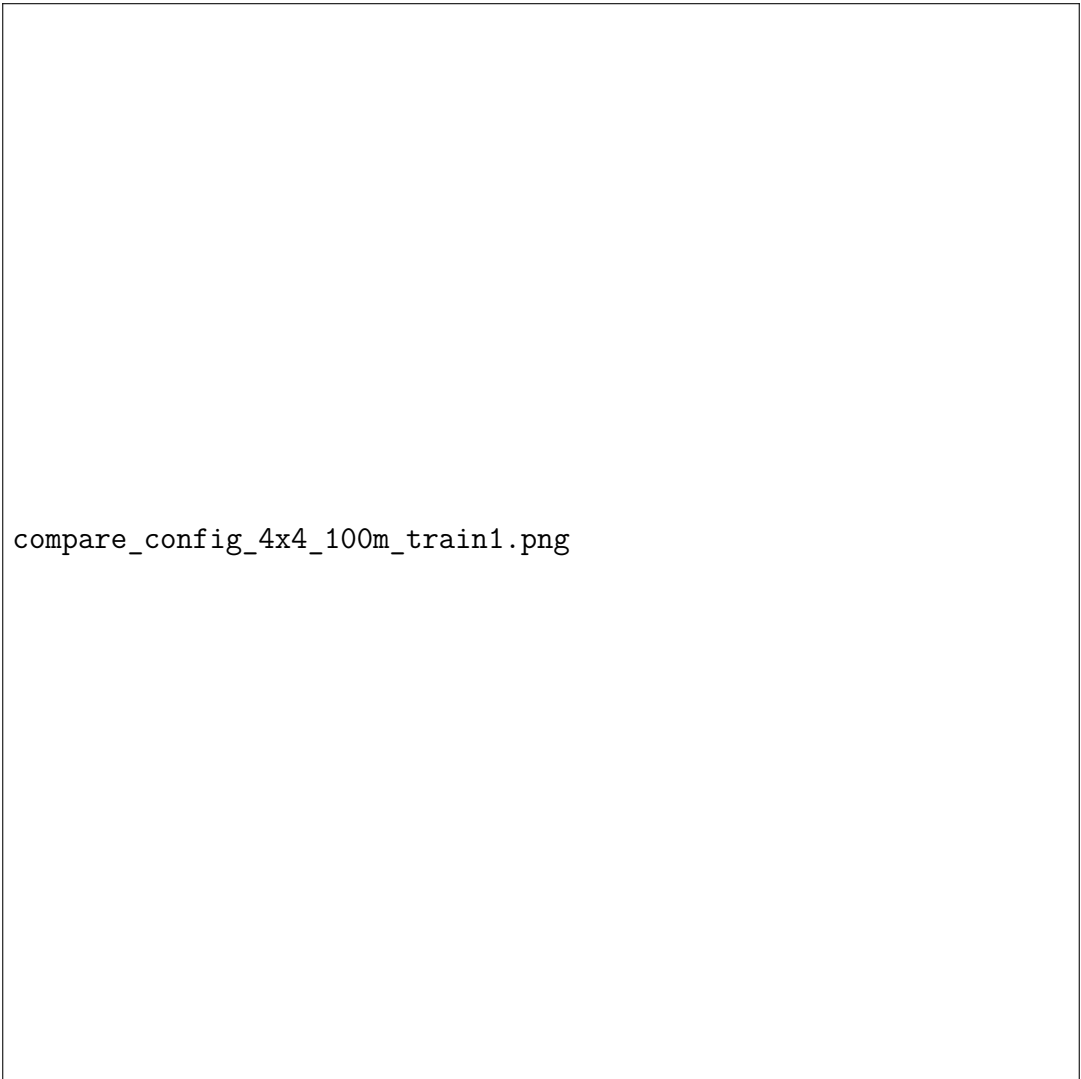
Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	% _{compl}
Fixed-Time	463.9	n.d.	79.8
MaxPressure	239.2	1635.0	88.5
MetaSTGAT Pro ($\alpha = 0.5$)	295.2	1142.1	86.8
MetaSTGAT Pro ($\alpha = 0.2$)	280.8	1103.6	87.6
MetaSTGAT Pro ($\alpha = 0.0$)	311.75	1607.1	84.6

di travel time che qualunque strategia informata (reattiva o appresa) dovrebbe poter migliorare. Tutti gli altri controllori lo battono nettamente (161.9-311.75s contro 339.3-463.9s), confermando che anche la regola più semplice tra quelle reattive (MaxPressure) coglie un segnale reale nello stato della rete.

MaxPressure resta il modello con il travel time medio più basso in entrambi i regimi, coerente con la sua ottimalità di throughput a livello di rete (Varaiya, 2013): nessuna delle varianti RL di questo lavoro la supera su questa singola metrica. All'interno dello sweep su α , $\alpha = 0.2$ ottiene il travel time medio più basso e, in entrambi i regimi, anche il $tt_{\max}$ più basso tra tutti i cinque modelli confrontati, MaxPressure incluso: una penalità anti-starvation moderata non solo non peggiora il caso medio rispetto a $\alpha = 0.5$, ma produce la coda più corta osservata in questo esperimento. $\alpha = 0.0$ (nessuna penalità) è invece la configurazione peggiore tra le tre varianti Pro su ogni colonna delle due tabelle: rimuovere il termine anti-starvation non libera il modello per ottimizzare meglio il throughput medio, lo peggiora anche su quello. Questo risultato è discusso più a fondo, insieme all'equità direzionale che lo motiva, in §4.4.

Tutti i modelli degradano passando dalle configurazioni di training a quelle di test, MaxPressure incluso (161.9→239.2s): una parte di questo divario riflette la maggiore difficoltà intrinseca delle configurazioni di generalizzazione (topologie più grandi, densità di picco), non solo un limite di generalizzazione specifico del controllo

appreso.



compare_config_4x4_100m_train1.png

Figura 4.1: Confronto su `config_4x4_100m_train1` (in-distribuzione): Fixed-Time, MaxPressure, MetaSTGAT Pro ($\alpha = 0.5/0.2$) e `ablation_temporal`. Manca solo $\alpha = 0.1$ (in training): figura da rigenerare includendolo, insieme ai preset di ablation restanti, a valle di questo capitolo.

4.4 Equità direzionale: il ruolo di α

Il divario è netto. Con la penalità anti-starvation al suo valore di riferimento ($\alpha = 0.5$), l'attesa massima direzionale è 2-3 volte più bassa di MaxPressure in entrambi i regimi; a $\alpha = 0.2$ resta comunque molto più bassa (516-566s contro 1408-1494s in test), pur essendo leggermente peggiore di $\alpha = 0.5$ su tre delle quattro colonne. A $\alpha = 0.0$, l'equità direzionale collassa verso valori vicini a quelli di MaxPressure (1275-1530s in training e nella direzione N/S del test, paragonabile o peggiore su W/E in

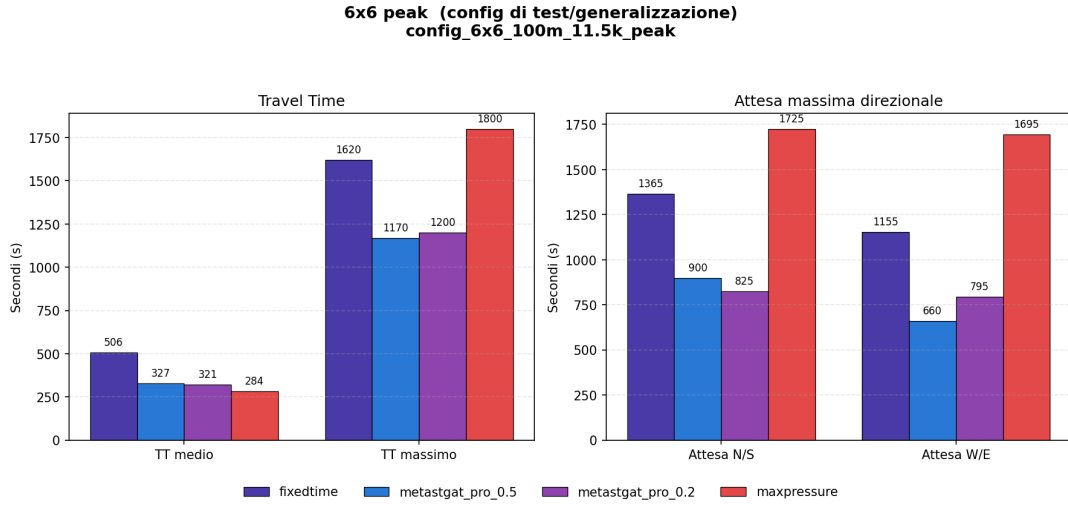


Figura 4.2: Confronto sulla configurazione di generalizzazione più difficile (6×6 , densità di picco), stessi sei controllori di Figura 4.1.

Tabella 4.4: Attesa massima direzionale (N/S e W/E), mediata sulle configurazioni di training e di test rispettivamente.

Modello	Training (s)		Test (s)	
	N/S	W/E	N/S	W/E
MaxPressure	755.0	825.0	1407.9	1493.6
MetaSTGAT Pro ($\alpha = 0.5$)	300.0	345.0	591.4	546.4
MetaSTGAT Pro ($\alpha = 0.2$)	455.0	450.0	516.4	565.7
MetaSTGAT Pro ($\alpha = 0.0$)	1275.0	1130.0	1530.0	1133.6

training): esattamente il comportamento atteso rimuovendo dalla ricompensa l'unico termine che penalizza le attese prolungate sulle corsie rosse.

Il punto di correttezza sperimentale già anticipato in §4.2 si applica direttamente qui: ciascuno di questi numeri è l'esito di un'unica realizzazione deterministica per configurazione, mediata su un piccolo numero di configurazioni (3 o 7). L'effetto di α sull'equità direzionale è ampio e consistente in direzione su tutte le configurazioni osservate (non un singolo caso isolato), il che lo rende un risultato qualitativamente solido; non è tuttavia accompagnato da un test di significatività statistica in senso classico, che richiederebbe più istanze indipendenti della stessa configurazione, non disponibili in un simulatore deterministico senza generare varianti a seed multiplo aggiuntive (§4.2, nota sul determinismo).

Il fatto che $\alpha = 0.0$ sia anche il peggiore, non il migliore, su travel time medio e massimo (Tabelle 4.2-4.3) esclude la spiegazione più semplice di un compromesso equità-contro-throughput: in questi dati, il termine anti-starvation non compra equità al prezzo del throughput, sembra piuttosto aiutare entrambi contemporaneamente, quantomeno nell'intervallo $\alpha \in [0.2, 0.5]$ esplorato qui. Una possibile lettura è che una rete dove nessuna corsia resta bloccata a tempo indefinito fluisce meglio nel suo complesso, non solo per i veicoli sulla corsia sfavorita: la verifica di questa ipotesi richiederebbe un'analisi per-corsia o per-fase che esula dallo scopo di questo capitolo.

4.5 Studio di ablation: risultati preliminari

Al momento della stesura di questo capitolo è disponibile un solo confronto completo del sistema di ablation a preset (Tabella 4.1): `temporal` (BPTT e burn-in sulla Meta-LSTM disattivati) contro Pro allo stesso $\alpha = 0.5$, l'unica variabile che li distingue. I quattro preset restanti (`paper`, `environment`, `rl_core`, `replay_stability`) sono in addestramento; questa sezione va integrata con i loro risultati non appena disponibili, senza modificarne l'impostazione.

Tabella 4.5: MetaSTGAT Pro ($\alpha = 0.5$) contro `ablation_temporal` (BPTT e burn-in disattivati), stesso α .

Modello	Training		Test	
	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\overline{tt}$ (s)	$tt_{\max}$ (s)
Pro ($\alpha = 0.5$)	205.5	800.0	295.2	1142.1
<code>ablation_temporal</code>	220.1	865.0	350.2	1375.7

Disattivare BPTT e burn-in peggiora il travel time medio su entrambi i regimi (+7.1% in training, +18.6% in test) e allarga anche la coda ($tt_{\max}$ +8.1% e +20.5% rispettivamente). Il fatto che il peggioramento relativo sia maggiore in test che in training è un indizio, non una prova data la numerosità limitata, che il disallineamento dello stato ricorrente corretto da BPTT+burn-in (§2.1, *hidden state staleness*) non sia solo un problema di adattamento alla distribuzione di training, ma pesi anche sulla capacità del modello di generalizzare a topologie mai viste: un’ipotesi coerente con la motivazione originale del meccanismo (Kapturowski et al., 2019), che richiederebbe però il confronto con gli altri preset (in particolare `rl_core` e `replay_stability`) per essere isolata con maggiore sicurezza dal contributo di Double DQN, PER e delle altre differenze architetturali che `temporal` lascia invece attive.

Bibliografia

- Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Alessandro Trenta, Alessio Gravina, and Davide Bacciu. Sonar: Long-range graph propagation through information waves. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.
- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Min Wang, Libing Wu, Man Li, Dan Wu, Xiaochuan Shi, and Chao Ma. Meta-learning based spatial-temporal graph attention network for traffic signal control. *Knowledge-Based Systems*, 250:109166, 2022. doi: 10.1016/j.knosys.2022.109166.
- Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haili Jin, and Zhenhui Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW)*, 2019. doi: 10.1145/3308558.3314139.